

컴퓨터 그래픽스 책

1장 : 컴퓨터 그래픽스의 개관

1-1. 컴퓨터 그래픽스의 개념과 활용

컴퓨터 그래픽스 :

- 컴퓨터에 의한 그림 자료의 표현과 조작
- 그림 자료를 생성하고 조작하기 위해 사용되는 여러 가지 기술
- 디지털 기법을 이용하여 시각적 콘텐츠를 합성하고 조작하기 위한 기술을 연구하는 컴퓨터과학의 분야

CAD(Computer-Aided Design)

디지털 프로토타이핑

컴퓨터 환경을 통해 제품을 설계하고 기능이나 동작을 시뮬레이션하는 드루이 절차를 수행하는 제품개발 방법

디지털 아트

컴퓨터에서 디지털의 형태로 만들어진 예술

컴퓨터 예술

컴퓨터로 만들어진 작품을 일컬을 때 사용

컴퓨터 영상을 생성하는 세 가지 주요한 방법

- 태블릿과 펜을 이용한 2D 컴퓨터 그래픽(래스터 그래픽스)
- 가상환경 창을 이용한 3D 컴퓨터 그래픽(벡터 그래픽스)
- 컴퓨터 프로그램을 이용한 2D나 3D 작품 생성

시각화 :

영상이나 도표, 애니메이션 등을 이용하여 정보를 제공하는 기술

가상현실

실제 세계를 시뮬레이션하거나 가상세계인 컴퓨터 모의환경과의 상호작용을 허용하는 기술

1-2. 컴퓨터 그래픽스 시스템의 구성

컴퓨터 그래픽스 시스템

- 그래픽스 워크스테이션 + 입출력을 위한 장치
- 워크스테이션 = CPU + 메모리 + 입출력 인터페이스 + GPU
- GPU : 그래픽스를 가속하기 위해 최적화된 전용 프로세서, 그래픽스 렌더링에 필수적인 실행산을 실행하기 위해 특별하게 설계
- 비디오 BIOS : 비디오 카드의 동작을 조절하고 컴퓨터와 소프트웨어가 비디오 카드와 상호 동작할 수 있도록 하는 명령어를 지원하는 기본 프로그램을 포함하는 펌웨어
- 비디오 메모리 : 화면에 출력할 그래픽 데이터를 저장하는 용도
- RAMDAC : CRT 디스플레이와 같이 아날로그 입력을 사용하는 컴퓨터 디스플레이 장치를 위해 디지털 신호를 아날로그 신호로 변환.

1-3. 컴퓨터 그래픽스 디스플레이

벡터 디스플레이

그림을 그래픽스 명령의 형태로 표현하는데(디스플레이 리스트), 리스트들이 재생 버퍼에 저장되어 그래픽스 프로세서가 초당 30회 이상 반복하여 디스플레이. 리스트 크기가 너무 크면 깜빡임이 발생.

```
MoveTo(0.6, 0.25)
LineTo(0.2, 0.7)
LineTo(0.8, 0.5)
```

래스터 디스플레이

- 도형 단위가 아닌 영사 프레임 단위의 출력을 하는 디스플레이 방식
- 전자빔이 왼쪽에서 오른쪽으로 주사되어, 한 줄의 전자빔을 생성. 해당 전자빔이 위에서 아래로 반복 그어지면서 영상을 표현.
- 반복을 얼마나 빨리 하냐가 '주사율'
- 화면에 출력되는 점들을 '픽셀'이라고 한다.
- 주사선당 픽셀 수 X 주사선 수 = '해상도'
- 해상도가 낮으면 그림을 정교하게 그릴 수 없는데, 직선을 그었을 때 거친 선이 그려지는 현상을 '에일리어싱'이라함. 이를 완화하기 위한 기술이 '안티에일리어싱'
- 주사변환, 레스터화 : 기하도형으로 구성된 그래픽스 객체를 이산적인 픽셀값의 집합으로 표현 처리 하는 것

디스플레이 장치

- 초기에는 CRT 모니터 주로 사용
- 2000년대 이후 평판 디스플레이가 기본으로 방사성/비방사성으로 나뉨
- 방사성 디스플레이 : 전기 에너지를 빛으로 변환. 플라즈마 패널, LED, OLED 등이 예

- 비방사성 디스플레이 : 주변의 빛을 반사광이나 백라이트 등 다른 광원을 이용. LCD
- 해상도 : 분해능이라고도 하며, 픽셀 수로 표현
- 도트 피치 : 인접한 두 화소의 중심 사이의 거리 작으면 좀 더 선명한 영상 표현. 일반적인 경우 수평/수직 피치값이 같다.
- 반응시간 : 화소의 색상이나 밝기를 변화시키는 데 필요한 최소 시간. LCD 모니터의 경우 btb, gtb로 측정된다. 반응시간이 16ms 이하이면 비디오 게임에 충분. 10ms 이하면 사람의 눈으로는 차이 없음.
- 재생률 : 1초 동안 데이터를 모니터에 표현하는 횟수. fps
- 지원 색상 : 얼마나 많은 색상을 지원하는가
- 시야각 : 시각적으로 수용가능한 최대 각도.
- 명암 대비 : 가장 밝은 밝기와 가장 어두운 밝기 사이의 비율
- 종횡비 : 스크린 가로/세로 비율

2장 : 컴퓨터 그래픽스 소프트웨어

2-1. 컴퓨터 그래픽스 영상의 표현

래스터와 벡터 그래픽스

컴퓨터 그래픽스 영상은 도형을 표현하는 방식에 따라 크게 래스터 그래픽스 영상과 벡터 그래픽스 영상으로 분류할 수 있다.

래스터 그래픽스 영상

- 사각형 격자 형태의 좌표에 맞추어 나열된 픽셀로 표현되며, 각 픽셀에는 그 위치에 해당하는 색이 지정된다.
- 장치독립적
- 래스터 영상의 크기는 영상을 이루고 있는 가로 픽셀 수와 세로 픽셀 수로 표현한다.
- 확대하면 흐릿한 영상이 된다.
- photoshop 등이 대표 툴.
- BMP, PNG, GIF, TIFF, JPEG 등

벡터 그래픽스 영상

- 점, 직선, 곡선, 다각형과 같은 기하학적 도형을 이용하여 그림을 표현
- 모든 픽셀을 저장하는 래스터와 다르게 시작점과 끝점 등 도형을 정의하는 정보만 저장하기 때문에 적은 용량으로도 표현할 수 있다.
- 확대해도 화질이 떨어지지 않음.
- 사진과 같이 음영이 매우 다양하고 불규칙한 형태로 변화하는 영상을 표현하는 데는 적합하지 않음.
- adobe illustrator가 대표 툴. 제작 이후 렌더링 과정을 통해 래스터 영상으로 변환.

- SVG, WMF, DXF, STL, VRML 등

2-2. 컴퓨터 그래픽스 소프트웨어

컴퓨터 그래픽스를 위한 소프트웨어는 특수목적 그래픽스 패키지와 범용 그래픽스 응용 프로그래밍 인터페이스(API)로 구분.

특수목적 그래픽스 패키지

- 다양한 응용 분야에 맞는 그림을 생성하기 위한 기능 및 편리한 사용자 인터페이스를 지원.
- 전문가가 아니어도 고품질 출력물 생성 가능.
- CAD, Photoshop, 3ds Max, Maya 등

범용 그래픽스 API

- 전문 프로그래머가 C/C++, Java 등과 같은 프로그래밍 언어에서 그래픽스 함수를 사용하여 프로그램을 작성할 수 있도록 하는 그래픽스 라이브러리.
- 장면의 설계 및 렌더링 작업을 처리하기 위한 최적화된 함수를 제공.
- 이러한 함수들은 CPU상에서 동작되는 소프트웨어로만 작성 될 수도 있고, GPU에 의해 가속되는 하드웨어의 지원을 받을 수 있다.
- 저수준과 고수준 그래픽스 API로 나눌 수 있음.

저수준 그래픽스 API

- 그림을 구성하는 점, 선, 다각형과 같은 그래픽스 기본 요소 및 이들에 대한 색상, 문양 등의 속성을 정의하고, 객체의 기하변환, 장면의 뷰잉 등 장면을 정의하여 이를 컴퓨터 화면에 표시하는 일련의 과정을 지시하는 함수들이 포함.
- GL, OpenGL, DirectX 등은 이러한 함수를 제공
- 장면을 만들고자 할 때는 장면 내의 물체를 구성하는 기본 요소를 개별적으로 정의하고, 이를 그리기 위한 세부적인 처리과정을 일일이 프로그램으로 작성해야 함.

고수준 그래픽스 API

- 장면 묘사를 위주로 하는 기능을 제공. 내부적으로 저수준 API를 이용하여 구현될 수 있음.
- 그래픽스 작업을 좀 더 단순하고 효율적으로 구현할 수 있도록 다양한 모형을 제공. 이를 새로운 모형으로 쉽게 변형할 수 있게 한다.
- 장면의 구성은 이러한 모형들로 구성된 객체를 계층적으로 조직화하여 처리. 프로그래머는 장면을 그리기 위해 세세한 처리과정을 나열하는 것을 피할 수 있음.
- Open Inventor(OpenGL에 기반을 둔 객체지향 3D 그래픽스 AI), VRML, Java 3D등이 예시

2-3. OpenGL 프로그래밍

OpenGL 개요

- 가장 널리 사용되는 그래픽스 라이브러리
- 다양한 플랫폼에서 활용할 수 있는 플랫폼 독립적 그래픽스 API
- 대체제로는 DirectX
- 저수준의 절차적 API로, 장면을 렌더링하기 위해 요구되는 정확한 단계를 프로그래머가 정확하게 규정하도록 요구.
- 서술적 API : 프로그래머는 장면에 대해 서술만 하고 구체적인 렌더링은 라이브러리가 관리.
- OpenGL 1.0 : 고정기능 하드웨어를 이용하여 고속화한 고정기능 파이프라인.
- OpenGL 2.0 : GPU를 이용한 유연한 처리 구조로 진화. 고정기능 파이프라인 대신 셰이더에 의해 동작하는 프로그램 가능 그래픽스 파이프라인을 사용. GLSL의 도입.
- GLSL : C와 유사한 문법으로 표현되는 명령을 사용하는 프로그래밍 언어로, 그래픽스 하드웨어의 동작을 제어하기 위한 셰이더 프로그램을 작성하는 데 사용된다.
- OpenGL 3.0 : 디프리케이션 메커니즘이 도입.
- 디프리케이션 : 어떠한 기능을 아프로는 사용하지 말도록 권고하는 것. 초기 OpenGL에서 흔히 사용되던 많은 함수가 디프리케이트되고 더욱 효율적인 함수로 대체

OpenGL의 라이브러리

- 프로그래머가 어떠한 확장 기능을 사용할 수 있는 지 관리 : GLEW(*), GLEE
- 입력과 출력을 도우는 윈도 시스템 : GLX(X), AppleGL, WGL(window)
- 장치에 독립적인 인터페이스 : GLUT, freeGLUT

OpenGL의 함수 형태

```
return_type <lib_prefix>FunctionName<arg_countn><arg_type>{v} (<argument>);  
{v}의 경우 인수가 벡터인 경우 첨가
```

- OpenGL 코어 라이브러리의 함수는 접두어로 'gl'로 시작. 명령어를 구성하는 단어의 첫 번째 문자는 대문자 사용

```
void glClear(GLbitfield buf);
```

- 상수의 정의는 'GL_'로 시작하고 모두 대문자로 표시. 단어 구분은 밑줄 사용.

```
glClear(GL_COLOR_BUFFER_BIT);
```

- 명령어 끝에는 몇 가지 접미사가 추가되기도 한다 (...2f 같이)
- OpenGL 자료형

OpenGL 데이터 타입	비트 크기	대응 C 언어 타입	접미사
GLbyte	8비트 정수	signed char	b
GLshort	16비트 정수	short	s
GLint, GLsizei	32비트 정수	long (int)	i
GLfloat, GLclampf	32비트 부동소수점	float	f
GLdouble, GLclampd	64비트 부동소수점	double	d
GLubyte, GLboolean	8비트 부호 없는 정수	unsigned char	ub
GLushort	16비트 부호 없는 정수	unsigned short	us
GLuint, GLenum, GLbitfield	32비트 부호 없는 정수	unsigned long (int)	ui

- 일부 OpenGL 명령어는 마지막 문자에 'v'를 사용할 수도 있는데, 이때 'v'는 여러 개의 값으로 표현되는 데이터를 일일이 인수로 전달하는 대신 배열에 저장하여 하나의 인수로 전달한다는 것을 의미한다.

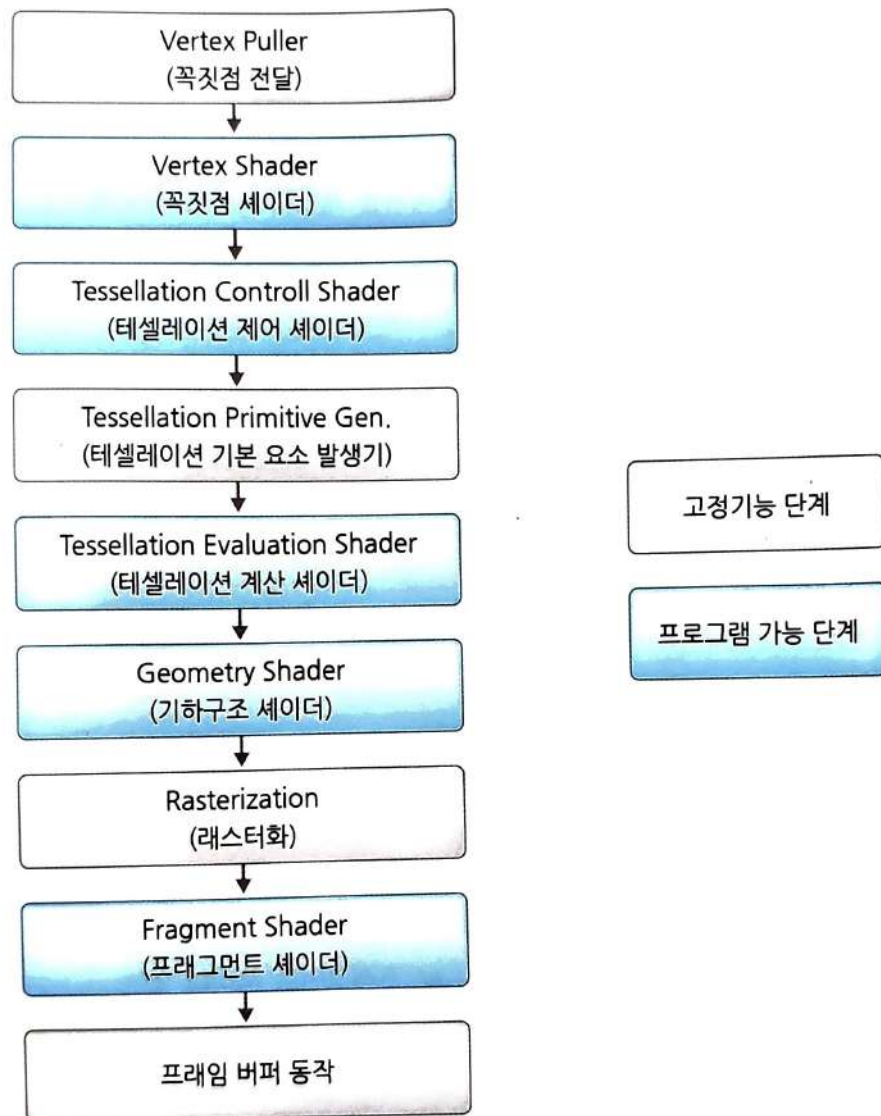
```
GLfloat fData = 0.5f, fData2 = 1.0f;
glUniform2f(vecLocation, fData1, fData2);

GLfloat fVec[2] = {0.5f, 1.0f};
glUniform2fv(vecLocation, fVec);
```

셰이더

- Open GL은 다음과 같은 여러 단계의 처리기가 파이프라인 형식으로 동작하게 되어 있다. 그중 일부는 고정기능 방식으로 동작하지만, 그 외의 처리기는 셰이더라는 프로그램에 의

해 동작.



[그림 2-2] OpenGL 파이프라인

- 꼭짓점 셰이더(vertex shader) : 꼭짓점 버퍼 객체를 통해 전달한 꼭짓점 및 관련 데이터에 대해 각각의 정점 단위로 프로그램에 의해 지정된 처리를 하여 다음 단계로 전달. 필수 단계
- 테셀레이션 셰이더(tessellation shader) : OpenGL 파이프라인 내에서 추가적으로 기하구조를 발생시킬 수 있는 단계. 선택적 단계로, 응용 프로그램에서 명시적으로 제공한 기하구조만 사용하도록 건너뛸 수 있음.
- 기하구조 셰이더(geometry shader) : 전체 기하구조 기본 요소를 수정할 수 있는 단계. 선택적
- 프래그먼트 셰이더(fragment shader) : 그래픽스 기본 요소를 프레임 버퍼에 그리기 이전 하나하나의 픽셀값을 구하는 것과 관련한 데이터를 의미. 이 데이터에는 프래그먼트의 래스터 좌표, 깊이 정보, 색, 알파, 텍스처 좌표 등이 포함. 래스터화 단계에서 발생된 개별 프래그먼트의 색 정보와 깊이 정보를 계산하여 후속 처리 단계로 보낸다. 필수 단계
- 셰이더는 GLSL을 이용하여 작성.

OpenGL 프로그램 예제

소스코드

[소스코드 2-1] OpenGLSample.c

```
1  #include <stdio.h>
2
3  #include <gl/glew.h>
4  #include <gl/freeglut.h>
5
6  typedef struct {          // 3차원 좌표를 표현하기 위한 구조체
7      float x, y, z;
8  } Vec3f;
9
10 enum { TRIANGLE, N_VBOs };
11 GLuint VBO[N_VBOs];      // 꼭짓점 버퍼 객체
12
13 static const char* pVS =  // 꼭짓점 셰이더 소스
14 "#version 330              \n"
15 "layout(location = 0) in vec3 Position; \n"
16 " \n"
17 "void main( )              \n"
18 "{                          \n"
19 "    gl_Position = vec4(Position*0.1, 1.0); \n"
20 "}";
21
22 static const char* pFS =  // 프래그먼트 셰이더 소스
23 "#version 330              \n"
24 "out vec4 FragColor;       \n"
25 " \n"
26 "void main( )              \n"
```

```

27     "{\n"
28     "    FragColor = vec4(1.0, 0.0, 0.0, 1.0);\n"
29     "};\n"
30
31 void AddShader(GLuint shaderProg,
32               const char* pShaderSrc, GLint ShaderType)
33 { // 셰이더 생성
34   GLuint shader = glCreateShader(ShaderType);
35   if (!shader) {
36     fprintf(stderr, "오류 - Shader 생성(%d)\n", ShaderType);
37     exit(0);
38   }
39   // 셰이더 컴파일
40   const GLchar* src[1] = { pShaderSrc };
41   const GLint len[1] = { strlen(pShaderSrc) };
42   glShaderSource(shader, 1, src, len);
43   glCompileShader(shader);
44   GLint success;
45   glGetShaderiv(shader, GL_COMPILE_STATUS, &success);
46   if (!success) { // 컴파일 오류 발생
47     GLchar infoLog[256];
48     glGetShaderInfoLog(shader, 256, NULL, infoLog);
49     fprintf(stderr, "오류 - Shader 컴파일(%d): %s\n",
50             ShaderType, infoLog);
51     exit(1);
52   }
53   // 셰이더 프로그램에 컴파일된 셰이더를 추가
54   glAttachShader(shaderProg, shader);
55 }
56
57 void SetUpShaders( )
58 { // 셰이더 프로그램 객체 생성
59   GLuint shaderProg = glCreateProgram( );
60   if (!shaderProg) {

```

```

59         fprintf(stderr, "오류 - Shader 프로그램 생성\n");
60         exit(1);
61     }
62
63     // 꼭짓점 셰이더 및 프래그먼트 셰이더 적재
64     AddShader(shaderProg, pVS, GL_VERTEX_SHADER);
65     AddShader(shaderProg, pFS, GL_FRAGMENT_SHADER);
66
67     GLint success = 0;
68     GLchar errLog[256];
69
70     glLinkProgram(shaderProg);      // 셰이더 프로그램 링크
71     glGetProgramiv(shaderProg, GL_LINK_STATUS, &success);
72     if (!success) {
73         glGetProgramInfoLog(shaderProg, sizeof(errLog), NULL, errLog);
74         fprintf(stderr, "오류 - Shader 프로그램 링크: %s\n", errLog);
75         exit(1);
76     }
77
78     glValidateProgram(shaderProg);  // 프로그램 객체가 유효한지 검사
79     glGetProgramiv(shaderProg, GL_VALIDATE_STATUS, &success);
80     if (!success) {
81         glGetProgramInfoLog(shaderProg, sizeof(errLog), NULL, errLog);
82         fprintf(stderr, "Invalid shader program: %s\n", errLog);
83         exit(1);
84     }
85     glUseProgram(shaderProg);      // 현재 셰이더 프로그램 객체로 지정
86 }
87
88 void RenderCB( )
89 {
90     glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
91     glClear(GL_COLOR_BUFFER_BIT); // 백색으로 화면 지움
92

```

```

93     glEnableVertexAttribArray(0);
94     glBindBuffer(GL_ARRAY_BUFFER, VBO[TRIANGLE]);
95     glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);
96     glDrawArrays(GL_TRIANGLES, 0, 3);
97     glDisableVertexAttribArray(0);
98     glFinish();
99 }
100
101 void InitVBOs()
102 {
103     Vec3f Vertices[3] = {        // 삼각형의 꼭짓점 좌표
104         {-5.0f, -5.0f, 0.0f},
105         {5.0f, -5.0f, 0.0f},
106         {0.0f, 5.0f, 0.0f} };
107     // 꼭짓점 버퍼를 생성하여 삼각형의 꼭짓점 좌표 전달
108     glGenBuffers(N_VBOs, VBO);
109     glBindBuffer(GL_ARRAY_BUFFER, VBO[TRIANGLE]);
110     glBufferData(GL_ARRAY_BUFFER, sizeof(Vertices), Vertices,
111                 GL_STATIC_DRAW);
112
113 }
114
115 int main(int argc, char** argv)
116 {
117     glutInit(&argc, argv);
118     glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
119     glutInitWindowPosition(50, 100);
120     glutInitWindowSize(640, 480);
121     glutCreateWindow("OpenGL Sample");
122
123     GLenum s = glewInit();
124     if (s != GLEW_OK) {
125         fprintf(stderr, "오류 - %s\n", glewGetErrorString(s));
126         return 1;
127     }

```

```
126
127     printf("GL version: %s\n", glGetString(GL_VERSION));
128     printf("GLSL version: %s\n",
            glGetString(GL_SHADING_LANGUAGE_VERSION));
129
130     SetUpShaders( );
131     InitVB0s( );
132
133     glutDisplayFunc(RenderCB);
134     glutMainLoop( );
135
136     return 0;
137 }
```

설명

- 이 프로그램은 크게 네 가지 작업 수행
 - OpenGL의 동작환경 준비 : OpenGL이 윈도우 시스템을 통해 사용자와 상호작용할 수 있도록 준비하고, 기능을 활용할 수 있도록 준비
 - 셰이더의 프로그램을 준비
 - 그림을 생성하기 위한 준비

- 실제로 그림을 만들어 낸다.

```

126
127     printf("GL version: %s\n", glGetString(GL_VERSION));
128     printf("GLSL version: %s\n",
129           glGetString(GL_SHADING_LANGUAGE_VERSION));
130
131     SetUpShaders( );
132     InitVB0s( );
133
134     glutDisplayFunc(RenderCB);
135     glutMainLoop( );
136
137     return 0;
138 }

```

이 프로그램은 크게 네 가지 작업을 수행한다.

첫째, OpenGL의 동작환경을 준비한다. 이러한 작업에는 OpenGL이 윈도우 시스템을 통해 사용자와 상호작용할 수 있도록 준비하고, OpenGL의 기능을 활용할 수 있도록 준비하는 것이 포함된다.

둘째, 셰이더의 프로그램을 준비한다.

셋째, 그림을 생성하기 위한 준비를 한다. 이 프로그램에서는 그림을 구성하는 물체, 즉 삼각형의 꼭짓점 데이터를 준비한다.

넷째, 실제로 그림을 만들어 낸다.

(1) OpenGL 동작환경 준비

이 프로그램에서는 윈도우를 통한 사용자와의 인터페이스를 제공하기 위해 freeGLUT를, OpenGL의 확장기능 관리를 위해 GLEW를 사용한다. [소스코드 2-1]의 3, 4행은 이 API를 사용하기 위한 헤더 파일을 포함시키고 있다.

프로그램이 윈도우 시스템을 통해 그림을 출력하거나 사용자와 상호작용할 수 있도록 준비하기 위해 먼저 115행에서 다음과 같이 함수 glutInit를 호출하여 GLUT를 초기화한다.


```
glutInit(&argc, argv);
```

이 초기화 함수는 명령행 인수도 처리할 수 있지만, 이 프로그램 예에서는 이러한 파라미터를 사용할 필요가 없을 것이다.

디스플레이 방식의 옵션을 116행과 같이 `glutInitDisplayMode` 함수를 사용하여 지정할 수 있다. 이 루틴에 전달하는 옵션 파라미터 상수들은 GLUT 심벌 상수로 지정된다. 예를 들어, 다음 문장은 디스플레이 윈도우에 하나의 재생 버퍼가 사용되고, 색 지정에 RGB(적·녹·청) 색 모드가 사용되도록 지정한다.

```
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
```

이 함수에 전달되는 상수들은 논리합 연산을 이용하여 결합된다. 사실 단일 버퍼링과 RGB 색 모드는 디폴트 옵션이다. 그러나 이러한 옵션이 디스플레이를 위해 사용되는 것을 상기하도록 이 함수를 사용할 것이다. 그 밖에도 애니메이션 응용을 위한 이중 버퍼링, 3차원 장면의 뷰잉을 위한 파라미터의 선택과 같은 디스플레이 옵션을 이 함수를 사용하여 지정할 수 있다.

디스플레이 윈도우는 디폴트 위치에 디폴트 크기로 만들 수도 있지만, GLUT 함수를 추가로 사용하여 이러한 파라미터들을 지정할 수도 있다. 디스플레이 윈도우의 좌상단 모서리의 초기 위치는 117행의 `glutInitWindowPosition` 함수를 사용하여 지정할 수 있다. 이 위치는 정수 스크린 좌표로 지정되는데, 원점은 스크린의 좌상단 모서리가 된다. 예를 들어, 다음 문장은 디스플레이 윈도우의 좌상단 모서리가 스크린 좌측 경계로부터 우측으로 50픽셀, 상단 경계로부터 아래로 100픽셀 떨어진 곳에 위치하도록 지정한다.

```
glutInitWindowPosition(50, 100);
```

마찬가지로 118행의 `glutInitWindowSize` 함수는 디스플레이 윈도우의 초기 픽셀 폭과 높이를 지정하는 데 사용된다. 초기의 폭이 640픽셀, 높이가 480픽셀인 디스플레이 윈도우를 다음 문장으로 지정한다.

```
glutInitWindowSize(640, 480);
```

디스플레이 윈도우의 위치나 크기는 윈도우가 생성되어 스크린에 디스플레이된 후 바꿀 수 있다.

다음은 디스플레이 윈도우를 생성한다. 윈도우 생성은 119행의 `glutCreateWindow` 함수를 사용하는데, 다음과 같이 제목 막대에 넣을 머리말을 파라미터로 전달한다.

```
glutCreateWindow("OpenGL Sample");
```

이제 디스플레이 윈도우는 생성되었다. 그러나 아직 화면에 윈도우가 디스플레이된 상태는 아니다. 먼저 윈도우를 다시 그릴 필요가 있을 때마다 반복적으로 호출하여 윈도우 안에 넣을 그림을 그릴 수 있는 함수를 알려야 한다. 이를 위해서는 프로그래머가 정의한 함수의 포인터를 GLUT에 등록해야 한다. GLUT는 이렇게 등록된 함수 포인터를 이용하여 필요할 때 이 함수를 호출한다. 이와 같은 형식으로 인수로 전달되는 함수를 콜백 함수(callback function)라고 한다. 윈도우를 디스플레이하기 위한 콜백 함수는 `glutDisplayFunc` 함수에 전달한다. 예를 들어, 콜백 함수로 앞으로 설명할 `RenderCB`라는 함수를 전달하고자 한다면 다음과 같은 문장을 사용한다.

```
glutDisplayFunc(RenderCB);
```

이제 윈도우를 디스플레이할 준비가 되었다. 다음은 OpenGL 확장기능 관리를 위한 GLEW의 초기화를 해야 한다. 이 작업은 GLUT의 초기화를 마친 후 해야 하며, 121행의 `glewInit` 함수를 사용한다. `glewInit` 함수는 정상적인 경우에는 `GLEW_OK`를 반환하지만, 만일 오류가 있다면 발생한 오류에 해당되는 값을 반환한다.

(2) 셰이더 프로그램

그림을 그리려면 먼저 셰이더 프로그램을 준비해야 한다. [소스코드 2-1]에서는 OpenGL의 프로그램 가능 파이프라인 단계 중 필수 셰이더인 꼭짓점 셰이더와 프래그먼트 셰이더를 포함하고 있다. GLSL로 작성한 셰이더 프로그램은 문자 배열에 넣어 OpenGL의 셰이더 컴파일 및 링크 함수 등을 이용하여 각 단계의 처리기에 전달한다. 간단한 셰이더 프로그램은 이 예에서처럼 OpenGL 소스 프로그램에 직접 포함시키기도 하지만, 별도의 파일에 작성한 다음, 이를 문자열 저장공간에 읽어 들인 후 처리하기도 한다.

꼭짓점 셰이더는 13~20행에서 문자열 상수로 작성하고 pVS가 이 꼭짓점 셰이더를 가리키도록 하였다.

14행은 '#'으로 시작하는데, C 언어와 같이 선행처리기 지시어 문장이다. 이 문장에서는 GLSL 버전 3.3을 사용하고 있음을 컴파일러에 알린다.

15행의 "in vec3 Position;"은 OpenGL 프로그램으로부터 전달되는 꼭짓점 정보가 vec3, 즉 3개의 GLfloat 값으로 표현되는 전역변수 Position에 입력됨을 나타낸다. 또한 "layout(location = 0)"는 꼭짓점 속성이 전달될 인덱스가 0번임을 알린다. 이것은 앞으로 언급할 glVertexAttribPointer 함수에서 0번 인덱스에 꼭짓점 좌표를 전달하는 것과 관계가 있다.

17~20행의 main 함수에서는 입력된 꼭짓점 좌표를 처리하여, gl_Position이라는 내장변수를 통해 다음 파이프라인 단계로 전달한다. 이 예에서는 Position에 전달된 좌표에 0.1을 곱하여 전달함으로써, 삼각형의 크기가 1/10로 축소되게 하였다. 단, gl_Position은 마지막에 동차 파라미터를 포함하여 4개의 GLfloat 값으로 표현되는 3차원 동차좌표이므로 동차 파라미터 1.0을 추가하여 vec4 데이터를 gl_Position에 넣는다. 동차좌표는 제3장에서 설명할 것이다. 여기서 꼭짓점 좌표를 1/10 축소한 것은 [그림 2-2]의 래스터화 단계에서 x, y, z 좌표가 모두 $[-1, 1]$ 범위에 있는 점들만 그리기 때문이며, 이 범위가 원도의 뷰포트에 맞추어 출력된다.

꼭짓점 셰이더가 동작하고 나면 처리된 삼각형의 세 꼭짓점 좌표가 파이프라인을 거쳐 래스터화 단계에 전달된다. 이제 이 좌표는 화면의 좌표로 변환되고, 삼각형을 구성하는 하나하나의 점들로 래스터화되어 프래그먼트 셰이더로 전달된다. 프래그먼트 셰이더의 기본적인 역할은 픽셀의 색을 결정하는 것이다.

프래그먼트 셰이더 역시 22~29행에서 문자열 상수로 작성하고 pFS가 이를 가리키도록 하였다. 24행에서는 색을 출력하기 위한 전역변수 FragColor를 선언하였다. R, G, B 및 알파값을 출력하기 위해 4개의 GLfloat 값을 나타내는 vec4로 선언하였다.

26~29행의 main 함수에서는 모든 프래그먼트에 대해 동일하게 $R=1.0, G=0.0, B=0.0, \text{알파}=1.0$ 을 출력한다. 그 결과, 삼각형은 빨강으로 그려질 것이다.

이제 이와 같이 준비된 셰이더 프로그램을 컴파일하고 링크하여 GPU에서 동작하도록 해야 한다. 55~86행의 함수 SetUpShaders는 이러한 작업을 수행한다. 먼저 57행의 glCreateProgram 함수로 프로그램 객체를 생성한 후, 이 객체에 37~53행의 AddShader 함수로 꼭짓점 셰이더(GL_VERTEX_SHADER)와 프래그먼트 셰이더(GL_FRAGMENT_SHADER)를

컴파일하여 셰이더 프로그램 객체에 첨부한다.

셰이더 프로그램을 컴파일하여 셰이더 프로그램 객체에 첨부하는 `AddShader` 함수에서는 먼저 33행의 `glCreateShader` 함수로 셰이더 객체를 생성한다. 셰이더 소스코드는 `glShaderSource` 함수로 셰이더 객체에 전달하는데, 셰이더 소스코드를 여러 개의 문자열로 나누어 만들고, 이들의 포인터를 저장하는 배열과 문자열의 길이를 저장하는 배열을 전달하도록 되어 있다. 이제 `glCompileShader` 함수에 셰이더 객체를 전달하여 컴파일한 후 `glGetShaderiv` 함수로 컴파일이 올바르게 되었는지 확인한다. 오류가 없다면 `glAttachShader` 함수로 셰이더 객체를 셰이더 프로그램 객체에 첨부한다.

모든 셰이더가 컴파일되면 70행의 `glLinkProgram` 함수를 이용하여 링크한 후, `glGetProgramiv` 함수를 이용하여 링크가 잘되었는지 확인한다. 만일 오류가 있다면 `glGetProgramInfoLog` 함수로 그 내용을 받아 출력한 후 프로그램을 중단시킨다. 링크가 되고 나면 `glValidateProgram` 함수로 프로그램을 검증한 후 `glUseProgram` 함수로 완성된 셰이더 프로그램을 사용하도록 설정한다.

(3) 그림 생성을 위한 준비

그림을 그리기 위해서는 그리려는 물체의 데이터를 준비해야 한다. `InitVBos` 함수에서는 그림을 만들기 위한 준비를 한다. 삼각형을 그리려면 삼각형의 세 꼭짓점 정보(좌표, 색 등)를 저장하는 버퍼가 필요하다. OpenGL에서는 꼭짓점 버퍼 객체(Vertex Buffer Object: VBO)를 통해 이러한 정보를 저장한다. VBO를 사용하려면 먼저 OpenGL의 `glGenBuffers` 함수로 VBO의 핸들(handle)을 생성해야 한다.

```
void glGenBuffers(GLsizei n, GLuint *buffers);
```

여기서 `n`은 생성할 VBO 핸들의 개수이고, `buffers`는 VBO 핸들을 저장할 배열이다. [소스코드 2-1]의 10행에서는 열거형을 이용하여 VBO의 번호에 이름을 부여하였다. 열거형은 선두의 이름부터 0번으로 시작하는 일련번호를 갖도록 이름에 번호를 부여한다. 따라서 우리가 만들려고 하는 삼각형에 지칭하는 이름인 `TRIANGLE`에는 0번이 부여되고, `N_VBos`에는 VBO의 개수에 해당되는 1이 부여된다. 11행에서는 VBO 개수의 크기에 해당되는 배열을 선언하고 있는데, 이 배열에 OpenGL에서 생성한 VBO의 핸들을 저장할 것이다.

VBO에 데이터를 전달하는 것은 OpenGL의 고유한 방법을 사용한다. 먼저 핸들을 생성한 후 `glBindBuffer` 함수로 이를 적절한 목적지 이름에 연결한다. 그다음 `glBufferData` 함수로 목적지 이름을 통해 데이터를 전달하면 그 데이터가 VBO에 저장된다.

```
void glBindBuffer(GLenum target, GLuint buffer);
void glBufferData(GLenum target, GLsizei size,
                  const void *data, GLenum usage);
```

여기서 `target`은 목적지 이름, `buffer`는 핸들, `size`는 바이트 단위의 데이터 크기, `data`는 전달할 데이터, `usage`는 데이터의 사용 형태를 나타낸다.

[소스코드 2-1]의 108행에서는 VBO 핸들 배열 VBO에 1개의 핸들을 생성한다. 109행은 삼각형의 VBO 핸들을 꼭짓점 속성에 해당되는 목적지를 나타내는 `GL_ARRAY_BUFFER`에 연결한다. 110행은 꼭짓점 데이터를 저장하고 있는 배열 `Vertices`를 삼각형의 VBO에 저장하는데, `GL_STATIC_DRAW`는 프로그램에서 VBO에 한 번 데이터를 지정한 후 변경하지 않는다는 것을 의미한다.

(4) 그림 생성

지금까지의 과정을 거쳐 그림을 생성할 준비가 되었다면 꼭짓점 데이터를 이용하여 그림을 생성할 수 있다. 그림을 생성하는 것은 88~99행의 `RenderCB` 함수를 사용한다.

90~91행은 프레임 버퍼를 흰색으로 지우도록 지시한다. `glClearColor` 함수는 R, G, B, 알파값을 전달받아 화면을 지울 색을 설정하고, `glClear` 함수는 지울 대상을 지정한다. 여기서는 `GL_COLOR_BUFFER_BIT`를 지정하여 프레임 버퍼가 지워지도록 하였다.

93행의 `glEnableVertexAttribArray` 함수는 꼭짓점 속성 배열의 인덱스를 활성화한다. 이 인덱스는 15행의 꼭짓점 셰이더에서 `location`의 값과 연결된다. 즉, 이 프로그램에서는 꼭짓점 셰이더에서 사용할 좌표(즉, `Position`)의 인덱스가 0번이며(layout (`location=0`)), 이 인덱스에 해당되는 꼭짓점 버퍼를 셰이더가 사용하는 것을 허용한다.

94행에서는 그림을 생성할 VBO를 다시 연결하였다. 이 프로그램에서는 1개의 VBO만 사용하기 때문에 매번 다시 연결할 필요는 없지만, 여러 개의 VBO를 사용하는 일반적인 경우에는 사용하고자 하는 버퍼를 연결하는 절차가 필요하다. `glVertexAttribPointer` 함수는 꼭짓점 속성 배열 인덱스에 해당되는 데이터를 꼭짓점 버퍼에서 어떻

게 액세스할 것인지 지정한다.

```
void glVertexAttribPointer(GLuint index, GLint size, GLenum type,
                           GLboolean normalized, GLsizei stride,
                           const GLvoid *pointer);
```

여기서 `index`는 속성의 인덱스를 나타내며(이 예에서는 0이며, 꼭짓점 좌표에 해당됨), `size`는 속성의 수(이 예에서는 속성이 x , y , z 좌표이므로 3개임), `type`은 속성의 자료형(이 예에서는 좌표값이 `GLfloat`이므로 `GL_FLOAT`임), `normalized`는 파이프라인에서 사용하기 전에 정규화를 할 것인지 여부를 나타낸다. `stride`는 연속적으로 이어지는 데이터 시작 위치 사이의 간격으로, 이 예에서는 12(즉, `sizeof(Vec3f)`)이다. 단, 이 예에서와 같이 꼭짓점 속성에 좌표값만 포함되어 데이터가 연속으로 이어지는 경우에는 단순히 0을 사용할 수 있다. `pointer`는 속성이 위치하는 데이터의 변위를 나타내며, 이 예에서는 버퍼의 제일 앞에서부터 데이터가 시작하므로 0이 사용되고 있다. 만일 1개의 꼭짓점마다 좌표 (x, y, z)와 색 (r, g, b)가 저장된다면 꼭짓점 좌표의 변위는 0, 그 뒤에 있는 색의 변위는 12가 될 것이다(단, '(const GLvoid*)12'와 같이 형변환을 해야 함). 이제 96행에서 `glDrawArrays` 함수로 그림을 생성하도록 지시한다.

```
void glDrawArrays(GLenum mode, GLint first, GLsizei count);
```

여기서 `mode`는 생성하려는 그래픽스 기본 요소를 지정하는 상수로, 이 예에서는 삼각형을 그리기 위해 `GL_TRIANGLES`를 사용하였다. `first`는 첫 번째 꼭짓점의 위치, `count`는 꼭짓점의 수이다.

그림 생성을 마치고 나면 `glDisableVertexAttribArray` 함수로 꼭짓점 속성 배열의 인덱스를 사용하지 못하게 한다. 그 결과, 그림을 생성하기 위한 모든 명령을 전달하였다. 그러나 이러한 명령은 즉시 실행되는 것이 아니라 OpenGL 서버에 전달되기 전에 명령 큐에 저장되어 차례로 실행된다. `glFlush`나 `glFinish` 함수는 이러한 실행을 제어하기 위한 것으로, `glFlush`는 가능한 한 빨리 명령들을 실행하도록 지시하는 반면, `glFinish` 함수는 이전에 큐에 전달한 명령들을 모두 실행한 후 복귀한다.

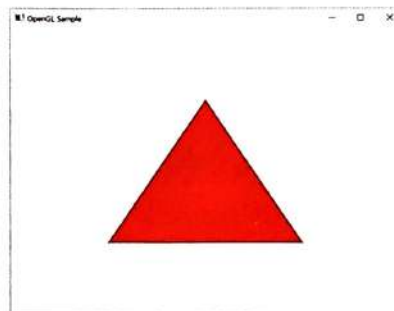
이처럼 그림을 생성하도록 작성한 `RenderCB` 함수는 명시적으로 호출하는 것이 아니라 앞서 언급한 것처럼 콜백 함수로 지정하여 윈도우의 이벤트에 맞추어 동작된다. 이

프로그램에서는 133행에서 `glutDisplayFunc` 함수를 통해 콜백 함수로 지정하였으므로, 윈도우가 만들어지거나 다시 그릴 필요가 있을 때마다 `RenderCB` 함수가 동작할 것이다. 이제 윈도우를 디스플레이할 준비가 되었다. 윈도우가 화면에 나타나도록 하기 위해서는 다음과 같이 GLUT의 메인 루프를 실행해야 한다.

```
glutMainLoop();
```

이 함수는 `glutDisplayFunc` 함수를 통해 전달된 콜백 함수에 따라 윈도우를 화면에 디스플레이한 후 그 안에 정의된 그림을 그린다. 그런 다음 윈도우에 발생하는 여러 가지 이벤트(마우스나 키보드 조작, 윈도우 크기 조정이나 최대화, 최소화 등)를 기다려 이에 해당하는 적절한 동작을 수행한다.

[그림 2-3]은 이 프로그램을 실행하여 화면에 디스플레이된 결과이다.



[그림 2-3] OpenGL에 의해 디스플레이된 윈도우